Classification: *K* nearest neighbor

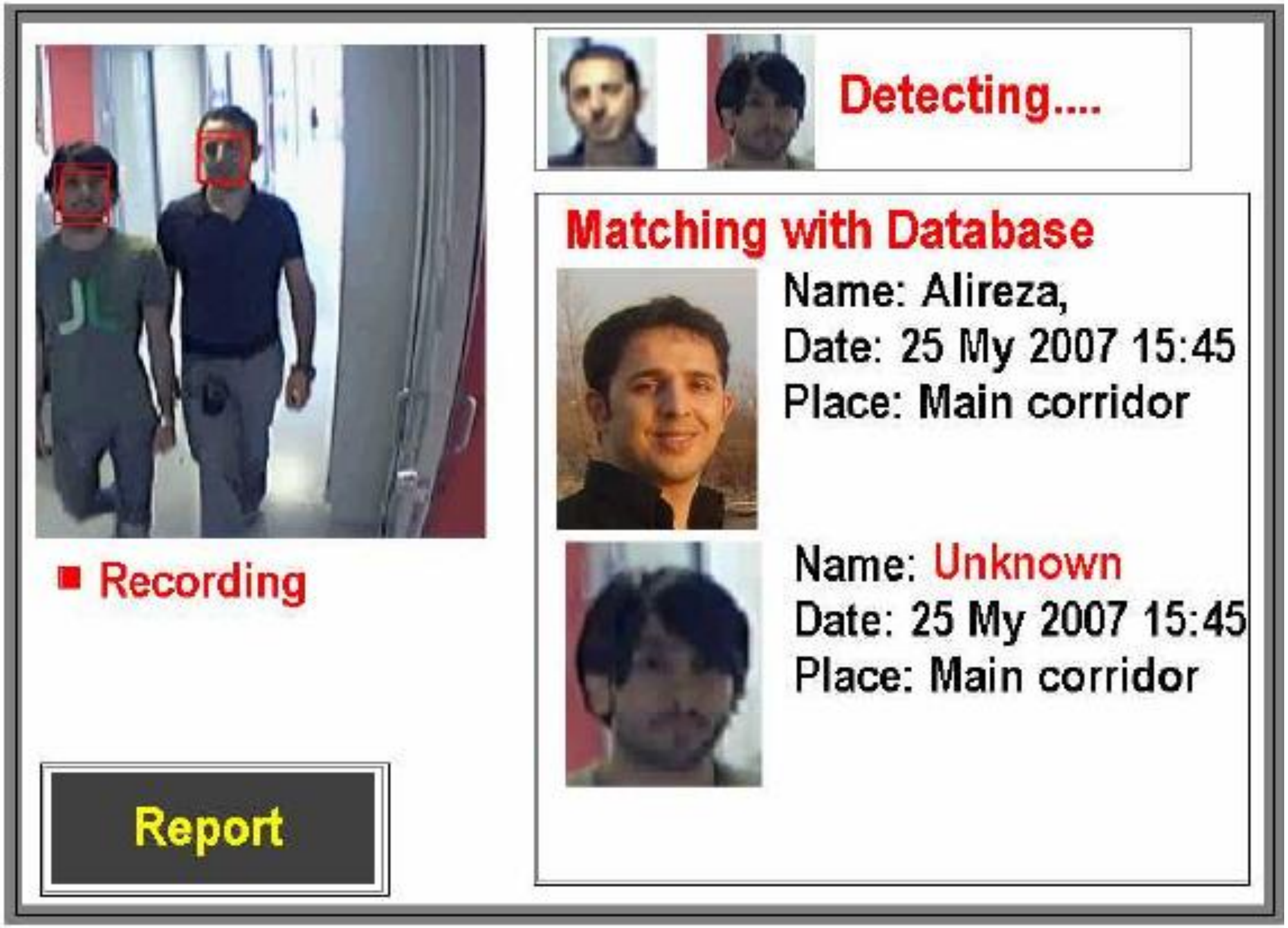
COMP7404
Computational Intelligence and Machine Learning

Dong Xu

Outline

- Face recognition: applications, challenges and systems
- *k* Nearest Neighbor classifier for face recognition

Application of Face Recognition in Video Surveillance



Labeled Faces in the Wild (2007)

Best results
without deep learning

Random guess
(50%)

MSRA TL Joint Bayesian (96.33

Human funneled (99.20%)

**CUHK deep learning result
(99.53%)**

**Google deep learning result
(99.6%)**

Applications of Face Recognition for Automatic Photo Organization

- Album organization: iPhoto 2009

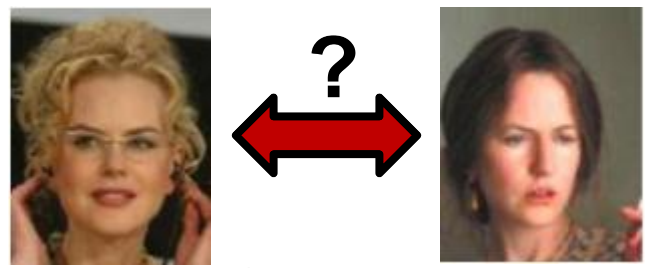
http://www.apple.com/ilife/iphoto/

Face Recognition Systems

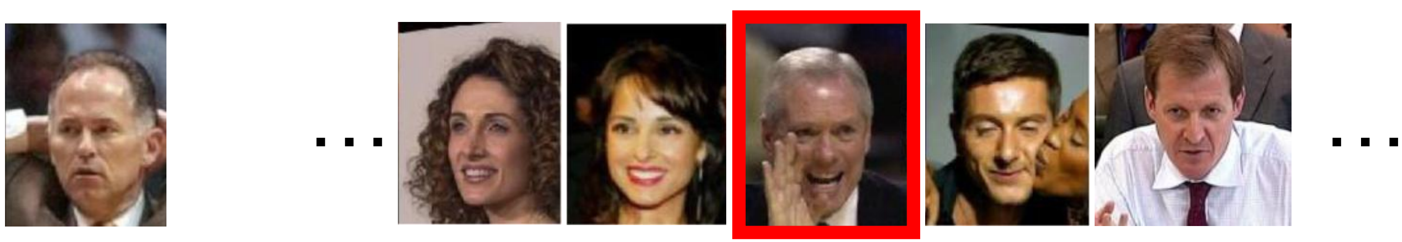
- Face detection
- Face alignment
- Face recognition

Typical face recognition scenarios

- Verification: a person is claiming a particular identity; verify whether that is true



- Closed-world identification: assign a face to one person from among a known set



- General identification: assign a face to a known person or to “unknown”

What makes face recognition hard?

- Expression
- Lighting
- Occlusion
- Viewpoint

Simple idea for face recognition

1. Treat face image as a vector of intensities

$\rightarrow \mathbf{x}$

2. Recognize face by nearest neighbor in database

$\rightarrow \mathbf{y}_1 \dots \mathbf{y}_n$

$$k = \underset{k}{\operatorname{argmin}} \|\mathbf{y}_k - \mathbf{x}\|$$

Classification: *k* Nearest Neighbor

- The *K* Nearest Neighbor (*kNN*) is a very intuitive method that classifies unlabeled examples based on their similarity to examples in the training set

    For a given unlabeled example $x_u \in R^d$, find the *k* "closest" labeled examples in the training data set and assign x_u to the class that appears most frequently within the *k*-subset.

- The *kNN* only requires

    1)An integer *k*

    2)A set of labeled examples (training data)

    3)A metric to measure "closeness"

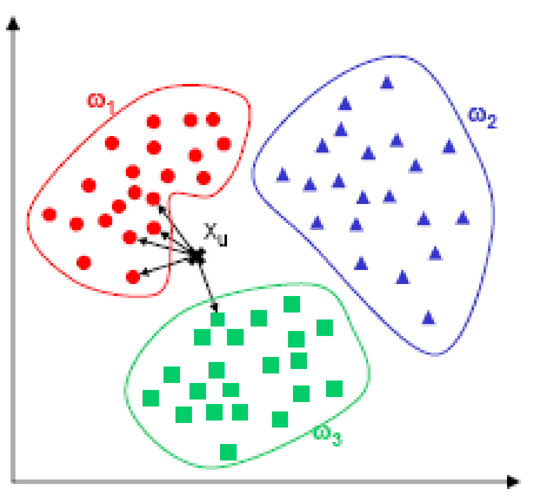
Classification: *k* Nearest Neighbor

- Example

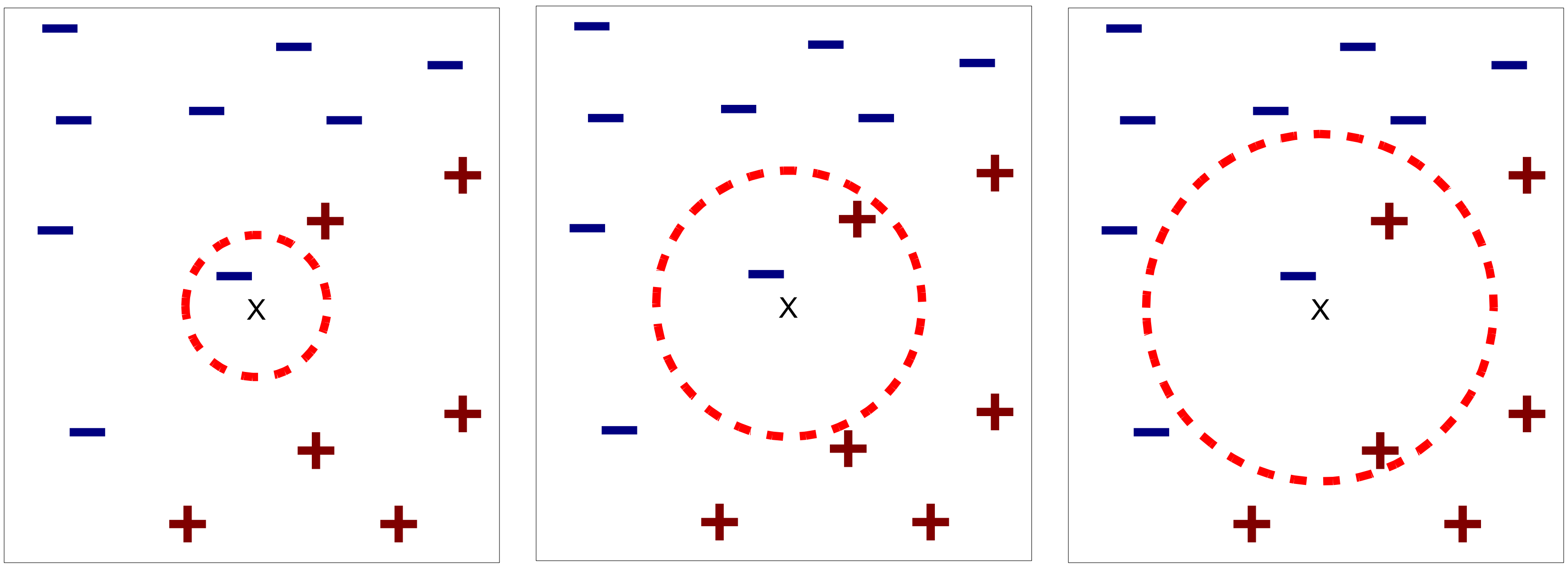
1)In the example below we have three classes and the goal is to find a class label for the unknown example x_u

2)In this case we use the Euclidean distance and a value of *k*=5 neighbors

3)Of the 5 closest neighbors, 4 belong to w_1 and 1 belongs to w_3 , so x_u is assigned to w_1 , the predominant class.



Definition of Nearest Neighbor



(a) 1-nearest neighbor (b) 2-nearest neighbor (c) 3-nearest neighbor

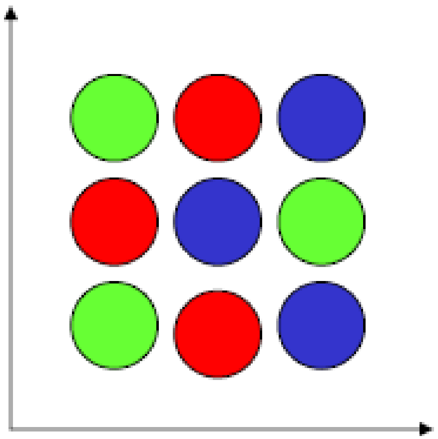
K-nearest neighbors of a record x are data points that have the k smallest distance to x

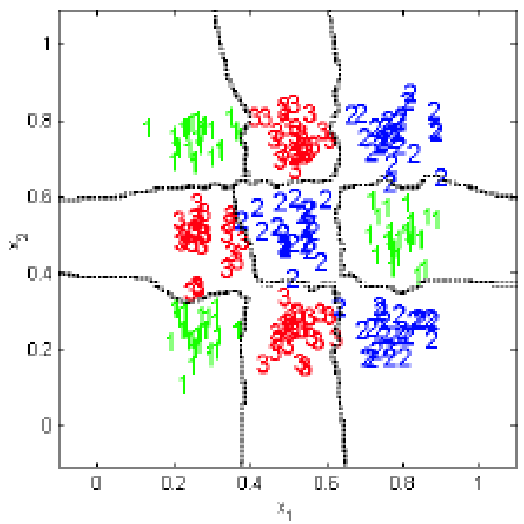
kNN : Example 1

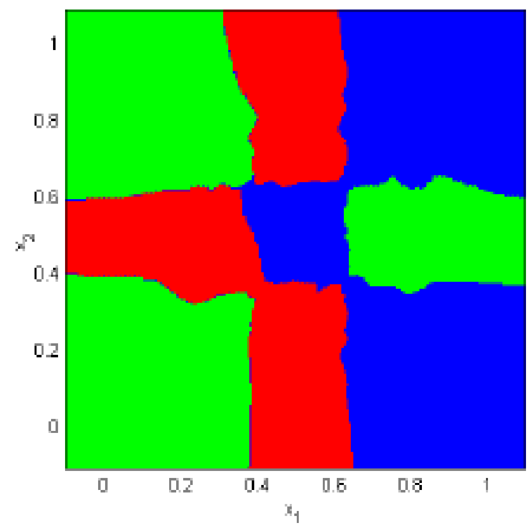
- We have generated data for a 2-dimensional 3-class problem, where the class-conditional densities are multi-modal, and non-linearly separable, as illustrated in the figure
- We used the kNN rule with

  1)k = 5; 2)The Euclidean distance as a metric

- The resulting decision boundaries and decision regions are shown below



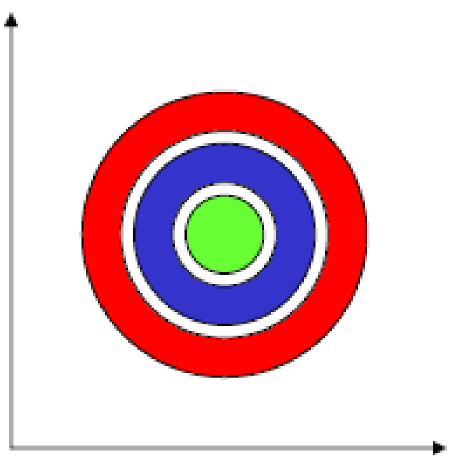


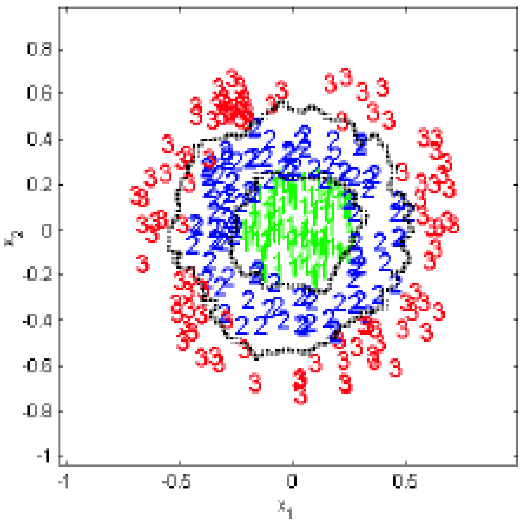


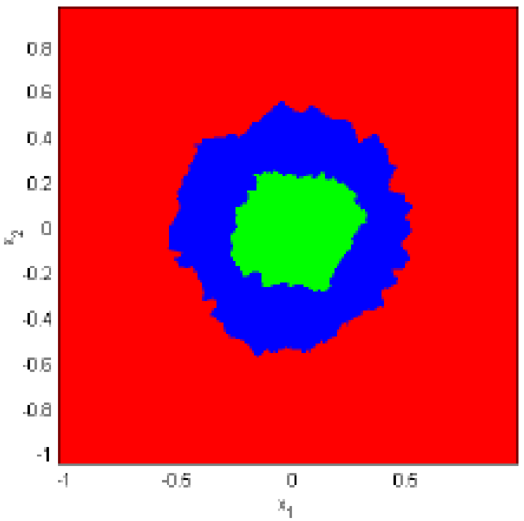
*k*NN: Example 2

- We have generated data for a 2-dimensional 3-class problem, where the class-conditional densities are unimodal, and are distributed in rings around a common mean. These classes are also non-linearly separable, as illustrated in the figure
- We used the *k*NN rule with

  1)k = 5; 2)The Euclidean distance as a metric
- The resulting decision boundaries and decision regions are shown below







*k*NN: A Lazy Algorithm

- *k*NN is considered a lazy learning algorithm

  1)Defers data processing until it receives a request to classify an unlabelled example

  2)Replies to a request for information by combining its stored training data

  3)Discards the constructed answer and any intermediate results

- This strategy is opposed to an eager learning algorithm which

  1) Compiles its data into a compressed description or model

     A density estimate or density parameters or a graph structure and associated weights

  2) Discards the training data after compilation of the model

  3) Classifies incoming patterns using the induced model, which is retained for future requests

Tradeoffs

  1)Lazy algorithms have fewer computational costs than eager algorithms during Training

  2)Lazy algorithms have greater storage requirements and higher computational costs on recall

Characteristics of the *k*NN classifier

- Advantages

1)Analytically tractable

2)Simple implementation

3)Nearly optimal in the large sample limit ($N \to \infty$)

4)Uses local information, which can yield highly adaptive behavior

5)Lends itself very easily to parallel implementations

- Disadvantages

1)Large storage requirements

2)Computationally intensive recall

3)Highly susceptible to the curse of dimensionality

*k*NN versus 1NN

- The use of large values of *k* has two main advantages

  a) Yields smoother decision regions

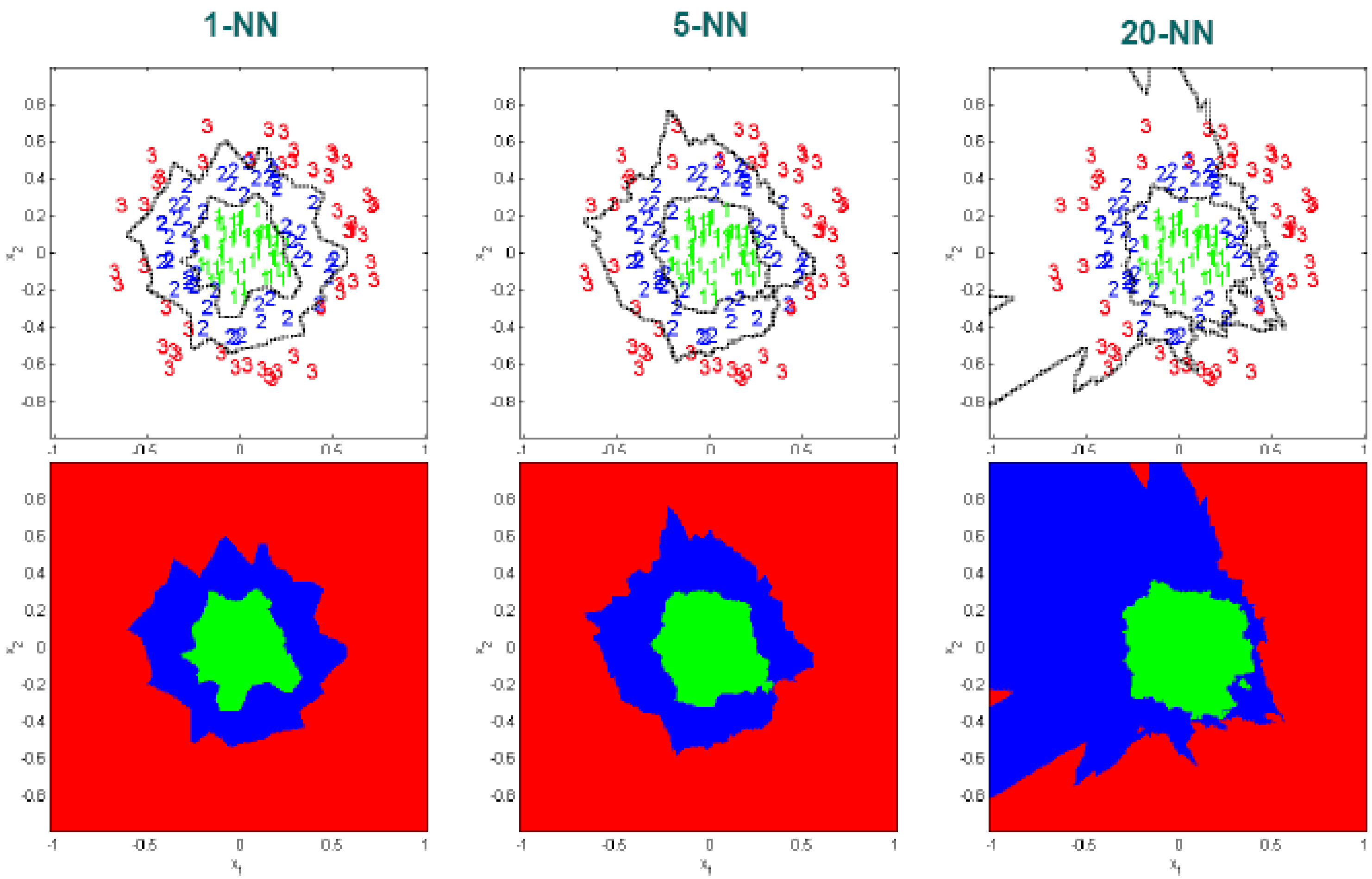
  b) Provides probabilistic information (The ratio of examples for each class gives information about the ambiguity of the decision)

- However, too large a value of *k* is detrimental

  a) It destroys the locality of the estimation since farther examples are taken into account

  b) In addition, it increases the computational burden

*k*NN versus 1NN



Optimizing storage requirements

- The basic *k*NN algorithm stores all the examples in the training set, creating high storage requirements (and computational cost)

  1) However, the entire training set need not be stored since the examples may contain information that is highly redundant

  2) In addition, almost all of the information that is relevant for classification purposes is located around the decision boundaries

- A number of methods, called edited *k*NN, have been derived to take advantage of this information redundancy

- A different alternative is to reduce the training examples to a set of prototypes that are representative of the underlying data

  clustering

Summary

- Face recognition: applications, challenges and systems
- *k* Nearest Neighbor classifier for face recognition

Thank you!